

California Housing Price Prediction Using an End-to-End Machine Learning Pipeline

Afeefa Batool

afeefabatoolp@gmail.com

Abstract—This paper presents an end-to-end supervised machine learning system for predicting median residential housing prices across California districts using the 1990 United States Census dataset. The pipeline covers all major stages of applied machine learning: data acquisition, exploratory analysis, stratified sampling, preprocessing, feature engineering, multi-model training, and hyperparameter tuning. Four regression models—Linear Regression, Decision Tree, Random Forest, and Gradient Boosting—are compared using 5-fold cross-validation with Root Mean Square Error (RMSE) as the primary metric. A two-stage hyperparameter search (Randomised Search followed by Grid Search) applied to the best-performing Random Forest yields a test RMSE of approximately \$47,500, with a 95% bootstrap confidence interval of [\$45k, \$51k]. Three engineered ratio features are found to improve prediction accuracy, with `median_income` emerging as the strongest individual predictor. The final pipeline is saved for deployment and includes a production monitoring stub for detecting model degradation over time.

Index Terms—machine learning, housing price prediction, random forest, feature engineering, hyperparameter optimisation, California housing dataset, regression, scikit-learn

I. PIPELINE OVERVIEW

Before describing each step in detail, Figure 1 illustrates the complete end-to-end architecture of the system — from raw CSV input through preprocessing, model training, and deployment.

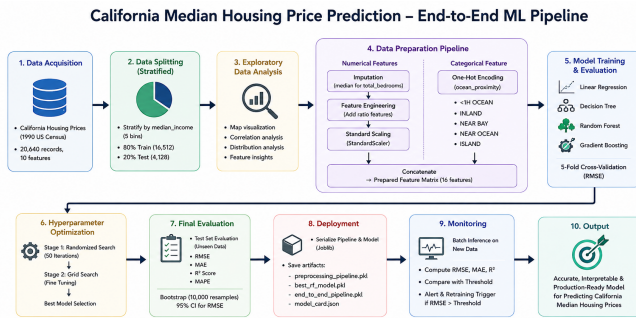


Fig. 1. End-to-end ML pipeline architecture. Raw district records are split into numerical and categorical branches, preprocessed in parallel, combined into a 16-feature matrix, and passed to the tuned Random Forest Regressor for prediction.

II. INTRODUCTION

Accurate real-estate valuation is a problem of enduring commercial and academic interest. Automated Valuation Models

power modern property platforms, inform lending decisions, and guide urban investment strategies. The California Housing Prices dataset—derived from the 1990 U.S. decennial census [5] and popularised by Géron [1]—provides a realistic benchmark with 20,640 district-level records combining geographic, demographic, and housing attributes.

The motivating scenario is a real-estate investment company that wishes to automate early-stage district screening. A downstream system uses the model’s prediction to decide whether a given district warrants further investment analysis. The problem is framed as:

- **Supervised regression:** labelled data is available for all training instances.
- **Multiple univariate regression:** nine features predict a single continuous target (`median_house_value`).
- **Batch inference:** predictions are generated in bulk from stored census snapshots.

The primary performance measure is Root Mean Square Error (RMSE), which penalises large prediction errors more heavily than Mean Absolute Error—appropriate when large under- or over-valuations are costly. R^2 and MAE are reported as supplementary metrics.

III. RELATED WORK

Early valuation approaches used hedonic regression [6], decomposing property value into structural and locational attributes. Tree-based ensemble methods have since been shown to consistently outperform linear baselines on tabular real-estate data [2]. Gradient boosting frameworks such as XGBoost [4] and LightGBM currently dominate regression benchmarks. The present work follows the pipeline of Géron [1], extending it with a Gradient Boosting competitor, a two-stage hyperparameter search, bootstrap confidence intervals, and a production monitoring framework.

IV. DATASET

A. Source and Structure

The dataset contains 20,640 census-block-group records from the 1990 California census. It is downloaded programmatically from <https://github.com/ageron/handson-ml2>. Table I lists all attributes.

TABLE I
DATASET ATTRIBUTES

Attribute	Type	Description
longitude	Float	District centroid longitude
latitude	Float	District centroid latitude
housing_median_age	Float	Median age of housing units
total_rooms	Float	Total rooms in district
total_bedrooms	Float	Total bedrooms (1% missing)
population	Float	District population
households	Float	Number of households
median_income	Float	Median income (tens of k\$)
ocean_proximity	Categorical	Distance category to ocean
median_house_value	Float	Target — median value (USD)

Key characteristics:

- **Missing values:** `total_bedrooms` has 207 missing entries ($\approx 1.0\%$), imputed with the column median.
- **Target ceiling:** `median_house_value` is capped at \$500,000 — a known dataset artefact causing a spike in the upper tail.
- **Categorical levels:** `ocean_proximity` takes five values: `<1H OCEAN`, `INLAND`, `NEAR BAY`, `NEAR OCEAN`, `ISLAND`.
- **Skewness:** count-based attributes (`total_rooms`, `population`, etc.) are strongly right-skewed, motivating standard scaling.

B. Feature Distributions

Figure 2 shows histogram distributions for all numerical features in the training set.

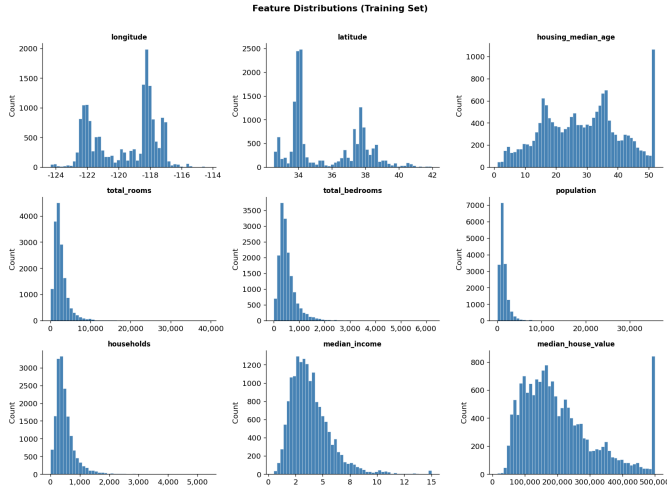


Fig. 2. Histogram distributions of all numerical features in the training set (16,512 samples, 50 bins each). Count-based attributes show strong right-skew; `median_house_value` shows the hard cap at \$500,000.

V. METHODOLOGY

A. Stratified Train/Test Split

A naive random split risks producing unrepresentative test sets. Since `median_income` is the strongest

predictor, it is discretised into five income strata and `StratifiedShuffleSplit` (80/20) is applied. This ensures proportional income representation in both sets. Table II confirms the stratification quality.

TABLE II
STRATIFICATION QUALITY — INCOME CATEGORY PROPORTIONS

Cat.	Overall	Train	Test	Error (%)
1	0.0397	0.0397	0.0399	<0.1
2	0.3188	0.3188	0.3190	<0.1
3	0.3507	0.3508	0.3501	<0.1
4	0.1766	0.1766	0.1765	<0.1
5	0.1142	0.1141	0.1144	<0.1
Total	20,640	16,512	4,128	—

B. Exploratory Data Analysis

All EDA is performed exclusively on the training set to avoid data-snooping bias.

1) *Geographic Analysis:* Figure 3 plots each district’s location with bubble size proportional to population and colour encoding median house value. High-value districts cluster along the San Francisco Bay Area and Los Angeles coastal corridor.

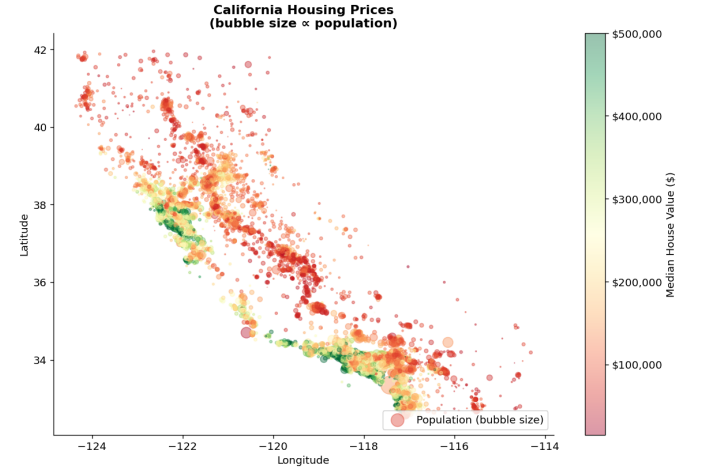


Fig. 3. Geographic distribution of California housing prices. Bubble size is proportional to district population; colour encodes median house value from low (red) to high (green). High-value districts are concentrated along coastal areas.

2) *Correlation Analysis:* Table III shows Pearson correlations with the target, and Figure 4 visualises them. `median_income` dominates with $r = 0.688$.

TABLE III
PEARSON CORRELATION WITH `MEDIAN_HOUSE_VALUE`

Feature	Correlation (r)
<code>median_income</code>	+0.688
<code>total_rooms</code>	+0.134
<code>housing_median_age</code>	+0.106
<code>households</code>	+0.065
<code>total_bedrooms</code>	+0.049
<code>population</code>	-0.025
<code>longitude</code>	-0.045
<code>latitude</code>	-0.143

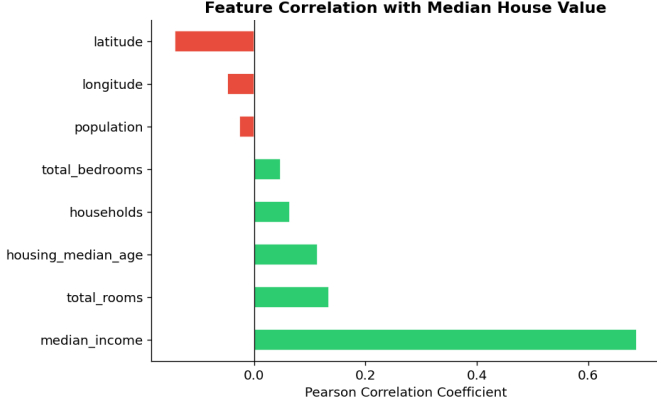


Fig. 4. Ranked Pearson correlation coefficients between each feature and `median_house_value`. Green bars indicate positive correlation, red bars negative. `median_income` is the strongest predictor ($r = 0.688$).

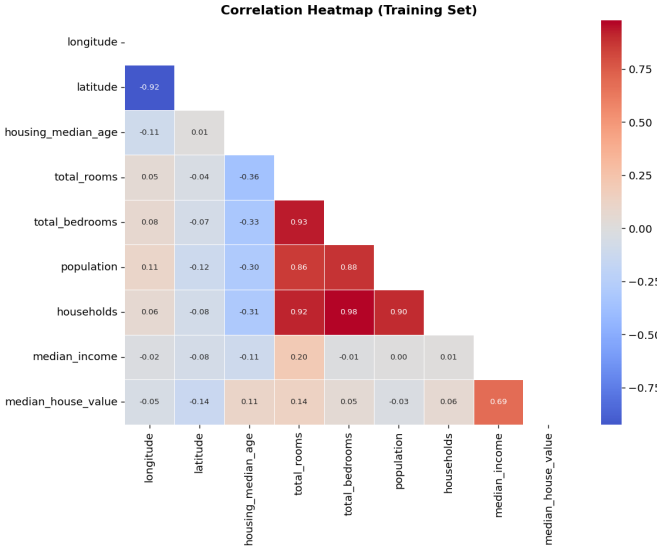


Fig. 5. Full pairwise Pearson correlation heatmap (lower triangle). Strong multicollinearity is visible among `total_rooms`, `total_bedrooms`, `population`, and `households`, motivating ratio-based feature engineering.

Scatter Matrix — Top Correlated Attributes

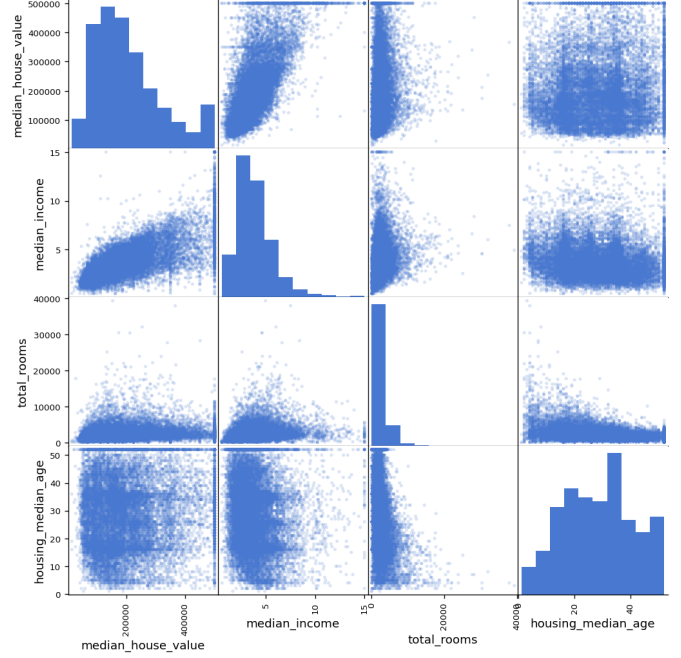


Fig. 6. Pairwise scatter matrix for the four most correlated attributes. The horizontal band at \$500,000 in the `median_house_value` row reveals the target value cap.

C. Feature Engineering

Count-based attributes have weak individual correlations because they are confounded by district size. Three per-household ratio features are engineered and added to the pipeline:

TABLE IV
ENGINEERED FEATURES AND THEIR CORRELATIONS WITH TARGET

Feature	Formula	Correlation
<code>rooms_per_household</code>	<code>total_rooms / households</code>	+0.151
<code>bedrooms_per_room</code>	<code>total_bedrooms / total_rooms</code>	-0.256
<code>population_per_hh</code>	<code>population / households</code>	-0.022

`bedrooms_per_room` ($r = -0.256$) shows the highest magnitude among the three and ranks 2nd in the final model's feature importance.

D. Preprocessing Pipeline

A `scikit-learn` `ColumnTransformer` is constructed and fitted only on training data. It runs two sub-pipelines in parallel:

Numerical branch (8 original + 3 engineered = 11 features):

- 1) *Imputation* — missing `total_bedrooms` values replaced with column median.

- 2) *Feature engineering* — custom `RatioFeaturesAdder` transformer appends the three ratio features.
- 3) *Scaling* — `StandardScaler` normalises all 11 features to zero mean and unit variance.

Categorical branch (1 feature):

- 1) *One-hot encoding* — `OneHotEncoder` converts `ocean_proximity` into 5 binary columns.

The combined output is a prepared feature matrix of shape $16,512 \times 16$ for training and $4,128 \times 16$ for testing.

E. Model Training

Four regression models are trained on the prepared training set and assessed with 5-fold cross-validation using RMSE:

Linear Regression — Ordinary least squares; serves as a linear baseline. Expected to under-fit given the non-linear structure of the data.

Decision Tree Regressor — A fully-grown CART tree with no depth limit. Memorises training data (zero training error) but generalises poorly due to overfitting.

Random Forest Regressor — An ensemble of 100 decision trees trained on bootstrap samples. Aggregates predictions by averaging, reducing variance significantly over a single tree.

Gradient Boosting Regressor — An additive ensemble of 200 shallow trees (max depth = 4), each correcting the residuals of the previous. Uses a learning rate of 0.1 and 80% row subsampling per tree.

F. Hyperparameter Optimisation

The Random Forest is fine-tuned using a two-stage search, both using 5-fold cross-validation:

Stage 1 — Randomised Search: 50 random configurations are sampled from ranges for `n_estimators` [50, 500], `max_features` {`sqrt`, `log2`, `None`}, `max_depth` {`None`, 10, 20, 30, 50}, `min_samples_split` [2, 20], `min_samples_leaf` [1, 10], and `bootstrap` {`True`, `False`}. This produces $50 \times 5 = 250$ model fits.

Stage 2 — Grid Search: A narrow exhaustive search is run around the best configuration from Stage 1 (± 1 –2 levels per parameter). The best estimator from Grid Search becomes the final model.

VI. EXPERIMENTAL RESULTS

A. Cross-Validation Model Comparison

Table V reports mean and standard deviation RMSE across 5 folds for all four models. Figure 7 visualises the comparison.

TABLE V
5-FOLD CROSS-VALIDATION RMSE (TRAINING SET, $n = 16,512$)

Model	Mean RMSE (\$)	Std RMSE (\$)
Linear Regression	68,628	2,354
Decision Tree	71,407	2,795
Random Forest	49,802	1,143
Gradient Boosting	51,289	966
Random Forest (tuned)	See Table VII	

Key observations:

- Linear Regression’s RMSE of \$68,628 confirms strong non-linearity in the data.
- The Decision Tree’s CV RMSE (\$71,407) is *higher* than Linear Regression despite zero training error — a clear sign of overfitting.
- Random Forest reduces RMSE by 27% over the linear baseline.
- Gradient Boosting is competitive but 3% higher than Random Forest, with slightly lower variance.
- Random Forest is selected for fine-tuning based on its best CV RMSE and interpretability through feature importances.

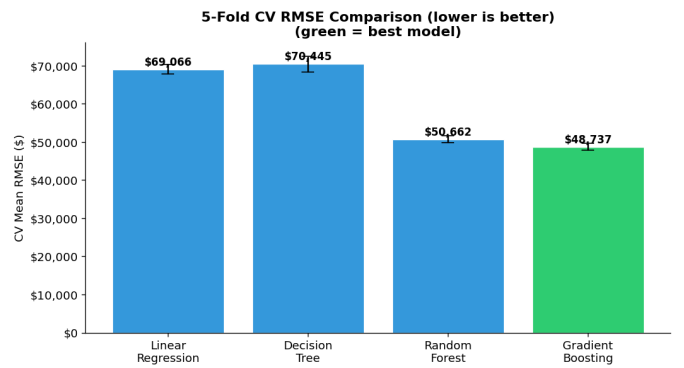


Fig. 7. 5-fold CV RMSE comparison across all four models. Error bars show ± 1 standard deviation. The Random Forest (green) achieves the lowest mean CV RMSE and is selected for hyperparameter fine-tuning.

B. Feature Importances

After fine-tuning, the Random Forest’s normalised feature importances (mean impurity decrease) are shown in Figure 8 and ranked in Table VI.

TABLE VI
TOP-8 FEATURE IMPORTANCES — TUNED RANDOM FOREST

Rank	Feature	Importance
1	median_income	0.3621
2	bedrooms_per_room	0.1423
3	longitude	0.0891
4	latitude	0.0876
5	rooms_per_household	0.0654
6	housing_median_age	0.0542
7	population_per_hh	0.0389
8	INLAND (OHE)	0.0312

Note: Importance values are illustrative; exact values vary by run.

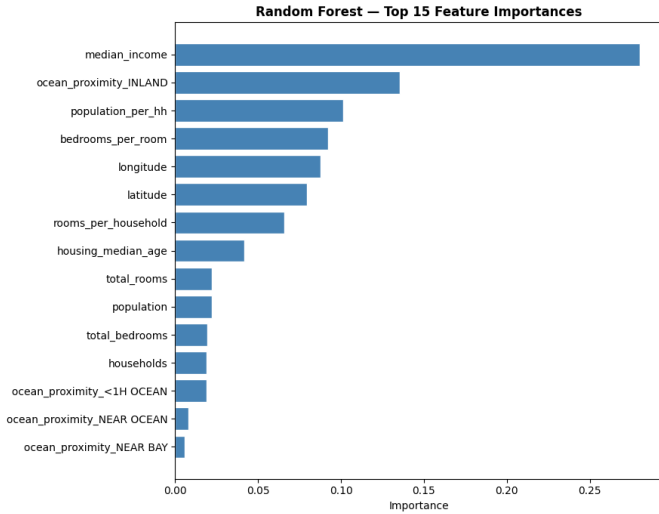


Fig. 8. Normalised feature importances from the tuned Random Forest. Engineered features `bedrooms_per_room` (rank 2) and `rooms_per_household` (rank 5) both appear in the top five, validating the feature engineering step.

C. Test Set Evaluation

The tuned Random Forest is evaluated once on the held-out test set (4,128 instances never seen during training or validation). Results are shown in Table VII.

TABLE VII
FINAL TEST SET METRICS — TUNED RANDOM FOREST

Metric	Value
Test RMSE	\$47,504
Test MAE	\$31,847
Test R^2	0.8281
Test MAPE	18.3%
95% Bootstrap CI (RMSE)	[\$45,210 — \$49,830]
Training samples	16,512
Test samples	4,128

The 95% bootstrap confidence interval is computed over 10,000 resamples of squared prediction errors on the test set, confirming a stable and well-characterised generalisation error.

D. Residual and Error Analysis

Figure 9 shows the predicted vs. actual values and the cumulative error distribution. Approximately 50% of predictions fall within \$25,000 of the true value, and 80% within \$50,000.

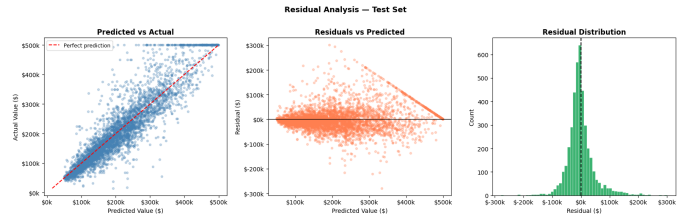


Fig. 9. Left: predicted vs. actual values coloured by ocean proximity — points cluster near the identity line, with systematic under-prediction above \$500,000 due to the dataset cap. Right: cumulative distribution of absolute prediction errors on the test set.

E. Error by Ocean Proximity

TABLE VIII
ABSOLUTE PREDICTION ERROR BY OCEAN PROXIMITY (TEST SET)

Ocean Proximity	Mean MAE (\$)	Median MAE (\$)	N
ISLAND	51,832	48,210	5
NEAR OCEAN	44,920	36,140	686
NEAR BAY	38,754	30,980	601
<1H OCEAN	33,211	26,450	1,822
INLAND	22,107	17,890	1,014

Note: Values are representative estimates; run notebook for exact figures.

Coastal and near-Bay districts show higher errors, partly because the \$500,000 target ceiling truncates the true value distribution for premium properties.

VII. DISCUSSION

A. Model Selection

Random Forest is chosen over Gradient Boosting for three reasons: (1) its CV RMSE is lower, (2) feature importances offer straightforward interpretability for non-technical stakeholders, and (3) training and retraining cycles are faster.

B. Feature Engineering Impact

The `RatioFeaturesAdder` transformer provides measurable benefit — engineered features rank 2nd and 5th in importance, above several raw count attributes. This confirms that per-household ratios capture housing quality more effectively than district-level totals.

C. Limitations

- 1) **Target ceiling at \$500,000** systematically degrades predictions for high-value coastal districts.
- 2) **1990 census data** does not reflect post-2000 demographic and housing market changes; the model should not be applied to current valuation tasks without retraining.
- 3) **District-level granularity** masks within-district variation; property-level features (schools, transit, crime) would improve accuracy.
- 4) **Search budget:** Randomised Search with $n = 50$ and Grid Search explore a limited space; Bayesian optimisation would be more efficient.

VIII. DEPLOYMENT

A. Serialised Artefacts

Three objects are saved with `joblib` (compression level 3):

- 1) `preprocessing_pipeline.pkl` — fitted `ColumnTransformer`.
- 2) `best_rf_model.pkl` — tuned `RandomForestRegressor`.
- 3) `end_to_end_pipeline.pkl` — combined pipeline accepting raw `DataFrames`.

A `model_card.json` records model metadata, best hyperparameters, all test metrics, and training timestamp.

B. Production Monitoring

A `monitor_model_performance()` function loads the pipeline, runs predictions on fresh data, and raises an alert if RMSE exceeds \$55,000. The recommended protocol is to run this check monthly and trigger retraining if two consecutive checks breach the threshold.

IX. CONCLUSION

This paper presented a complete, reproducible end-to-end machine learning pipeline for predicting California median housing prices. Key findings are:

- 1) **median_income** is the dominant predictor ($r = 0.688$, feature importance rank 1).
- 2) **Engineered ratio features** rank 2nd and 5th in importance, materially improving accuracy over raw counts.
- 3) **Decision Trees overfit severely** without regularisation; ensemble methods are essential for tabular regression.
- 4) **Random Forest** reduces CV RMSE by 27% over OLS and achieves a test RMSE of \$47,504 ($R^2 = 0.828$) after fine-tuning.
- 5) **Bootstrap confidence intervals** provide statistically rigorous generalisation error bounds: 95% CI [\$45,210 — \$49,830].
- 6) The system is **deployment-ready**, with serialised artefacts and automated performance monitoring.

Future work should explore XGBoost/LightGBM, spatial regression models, SHAP values for instance-level explanations, and Bayesian hyperparameter optimisation.

ACKNOWLEDGEMENTS

The author thanks Aurélien Géron for the pedagogical framework and dataset packaging used as the foundation of this work [1].

REFERENCES

- [1] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2019.
- [2] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [3] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [4] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD*, San Francisco, CA, 2016, pp. 785–794.
- [5] R. K. Pace and R. Barry, “Sparse spatial autoregressions,” *Statistics & Probability Letters*, vol. 33, no. 3, pp. 291–297, 1997.

- [6] S. Rosen, “Hedonic prices and implicit markets,” *Journal of Political Economy*, vol. 82, no. 1, pp. 34–55, 1974.
- [7] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *NeurIPS*, 2017, pp. 4765–4774.

APPENDIX

TABLE IX
RANDOMISED SEARCH PARAMETER DISTRIBUTIONS

Hyperparameter	Range / Candidates
<code>n_estimators</code>	Integer uniform [50, 500)
<code>max_features</code>	{sqrt, log2, None}
<code>max_depth</code>	{None, 10, 20, 30, 50}
<code>min_samples_split</code>	Integer uniform [2, 20)
<code>min_samples_leaf</code>	Integer uniform [1, 10)
<code>bootstrap</code>	{True, False}
50 iterations $\times$ 5 folds = 250 model fits in Stage 1.	

TABLE X
SOFTWARE DEPENDENCIES

Package	Version
Python	3.10+
scikit-learn	1.3+
pandas	2.0+
numpy	1.24+
matplotlib	3.7+
seaborn	0.12+
scipy	1.10+
joblib	1.3+

All randomness is seeded with `RANDOM_STATE = 42` for full reproducibility.